Innovation Centre for Education

NCS Nihon Communication Solutions

YENEPOYA (Deemed to be University)

Yenepoya Institute of Arts, Science, Commerce, and Management

Final Project Report

on

MULTI CLOUD HYBRID STRATEGY FOR SMART CITY

Student Name: Muhammed abdul gafoor

Industry Mentor: Mr. Sumit kumar shukla

Guided by: Ms.Deepthi k

Table of Contents

1. Executive Summary 3
2. Background 4
2.1 Aim 4
2.2 Technologies 5
2.3 Hardware Architecture 5
2.4 Software Architecture 6
3. System 8
3.1 Requirements 9
3.1.1 Functional Requirements 9
3.1.2 User Requirements 9
3.1.3 Environmental Requirements 10
3.2 Design and Architecture 10
3.3 Implementation 13
3.4 Testing 17
3.5 Graphical User Interface (GUI) Layout 19
3.6 Customer Testing 21
3.7 Evaluation 22
4. Snapshots of the Project 24
5. Conclusions 30
6. Further Development or Research 31
7. References 32
8. Appendix 33
8.1 Weekly Progress Reports 34

1. Executive Summary

The project titled "Automated Multi-Cloud Backup System with Smart City Issue Management" is designed to address two critical challenges faced by modern digital systems: efficient handling of user requests and secure data management. With the increasing reliance on digital platforms for managing services, it is essential to ensure both operational efficiency and data reliability.

This system integrates a complaint management platform with an automated backup mechanism. Users can submit complaints related to public services, which are then managed by administrators and assigned to relevant authorities. The system ensures transparency, accountability, and efficient tracking of all requests.

A major highlight of the system is its automated backup feature. Traditional backup systems require manual intervention and are prone to failure. In contrast, this system uses Django signals and scheduling mechanisms to automatically trigger backups whenever changes occur in the database. These backups are stored both locally and on cloud platforms such as Google Drive, ensuring redundancy and data safety.

The system follows a modular and scalable architecture, allowing easy integration of additional features. It leverages modern technologies such as Django, cloud APIs, and asynchronous processing to deliver high performance.

In conclusion, this project demonstrates a practical implementation of a hybrid system that combines issue management with automated multi-cloud backup, ensuring both operational efficiency and data security.

2. Background

In the digital age, the importance of reliable data management systems has grown significantly. Organizations, government institutions, and service providers rely heavily on digital platforms to store, process, and manage large volumes of data. However, these systems often face challenges such as data loss, system failures, and inefficient handling of user requests.

Traditional complaint management systems are often manual or semi-automated. These systems lack proper tracking mechanisms, leading to delays in resolving issues. Users are often unable to track the status of their complaints, resulting in dissatisfaction and lack of trust in the system.

Another major issue is the lack of reliable backup systems. Data loss can occur due to hardware failures, software bugs, cyberattacks, or human errors. Many systems rely on manual backup processes, which are time-consuming and prone to errors. In some cases, backups are not performed regularly, leading to permanent loss of critical data.

With the advancement of cloud computing, new opportunities have emerged for improving data storage and backup mechanisms. Cloud platforms provide scalable and secure storage solutions, enabling systems to store data in multiple locations. This reduces the risk of data loss and ensures high availability.

The concept of hybrid cloud systems combines local storage with cloud storage, providing the benefits of both. Local storage ensures fast access to data, while cloud storage provides redundancy and disaster recovery capabilities.

This project is developed in response to these challenges. It integrates a smart complaint management system with an automated multi-cloud backup mechanism. The system ensures

that all user data is securely stored, easily accessible, and protected against loss.

By combining modern web technologies with cloud-based solutions, the project aims to provide a reliable, efficient, and scalable system that can be used in real-world applications such as smart city management.

2.1 Aim

The primary aim of this project is to develop a comprehensive web-based system that integrates automated backup functionality with an efficient issue management platform. The system is designed to ensure data reliability, improve workflow efficiency, and provide a user-friendly interface for managing complaints.

The project aims to achieve the following:

- To design a scalable web application using Django framework
- To implement a structured complaint management system
- To automate the backup process using signals and schedulers
- To store backup data in both local and cloud environments
- To ensure quick recovery of data through restore mechanisms
- To provide role-based access for users, admins, and authorities
- To enhance system reliability through redundancy and failover mechanisms

The overall goal is to create a system that is both efficient and secure, capable of handling real-world challenges in data management and service tracking.

2.2 Aim

The primary aim of this project is to develop a comprehensive web-based system that integrates automated backup functionality with an efficient issue management platform. The system is designed to ensure data reliability, improve workflow efficiency, and provide a user-friendly interface for managing complaints.

The project aims to achieve the following:

- To design a scalable web application using Django framework
- To implement a structured complaint management system
- To automate the backup process using signals and schedulers
- To store backup data in both local and cloud environments
- To ensure quick recovery of data through restore mechanisms
- To provide role-based access for users, admins, and authorities
- To enhance system reliability through redundancy and failover mechanisms

The overall goal is to create a system that is both efficient and secure, capable of handling real-world challenges in data management and service tracking.

2.3 Technologies

The development of this system involves the use of various modern technologies that contribute to its functionality, performance, and scalability.

Python:
Python is the primary programming language used for developing the system. It is known for its simplicity, readability, and extensive library support.

Django Framework:
Django is a high-level Python web framework that follows the Model-

View-Template (MVT) architecture. It provides built-in features such as authentication, ORM, and security, making it ideal for web application development.

HTML, CSS, JavaScript:
These technologies are used for designing the frontend of the application. They ensure a responsive and user-friendly interface.

Database (MySQL/SQLite):
The system uses a relational database to store structured data. Django ORM is used to interact with the database.

Google Drive API (PyDrive2):
The PyDrive2 library is used to integrate Google Drive with the application. It allows uploading and managing backup files securely.

APScheduler:
This library is used to schedule backup tasks at regular intervals.

Threading:
Threading is used to execute backup operations asynchronously, ensuring that the main application is not affected.

Logging:
The logging module is used to track system activities and errors, helping in debugging and monitoring.

These technologies work together to create a robust and efficient system.

2.4 Hardware Architecture

The hardware architecture of the system is designed to support efficient execution and data processing. The system can run on standard computing hardware with moderate specifications.

Minimum Requirements:

- Processor: Intel i3 or higher
- RAM: 4GB or higher
- Storage: 500GB HDD or SSD
- Network: Stable internet connection for cloud access

The system is designed to be lightweight and can run on a local server or cloud-based server. For better performance and scalability, it can be deployed on high-performance servers with increased RAM and processing power.

The use of cloud storage ensures that hardware limitations do not affect data availability. Even if local hardware fails, data can be retrieved from the cloud.

2.5 Software Architecture

1. Presentation Layer (The "Face")

This is the only part of the system the user actually sees. In a Django environment, this is primarily composed of **Templates (HTML/CSS/JS)**.

- **Responsibility:** To capture user input (like a form submission) and display output (like a success message or a data dashboard).
- **Key Action:** It sends "requests" to the Application layer and waits for "responses." It does not know *how* the data is processed; it only knows how to show it.

2. Application Layer (The "Brain")

Often referred to as the **Business Logic Layer**, this is where your Django **Views and Forms** reside.

- **Responsibility:** This layer makes decisions. If a user uploads a file, the Application Layer checks: Is the file too large? Does the user have permission?
- **Coordination:** It acts as a traffic controller. It takes data from the Presentation layer, asks the Data layer to save it, and then tells the Integration layer to sync it to the cloud.

3. Data Layer (The "Library")

This layer is responsible for **Persistence**. In Django, this is handled by the **Models and the ORM (Object-Relational Mapper)**.

- **Responsibility:** It translates Python code into SQL commands that the database (like PostgreSQL or MySQL) can understand.

- **Abstraction:** Because of this layer, the rest of your app doesn't need to write complex database queries. You simply call `Model.objects.save()`, and the Data Layer handles the rest.

4. Integration Layer (The "Bridge")

Since your system connects to external platforms like **Google Drive API**, this layer acts as the translator between your app and the outside world.

- **Responsibility:** It manages API keys, handles authentication (OAuth), and formats data so external services can accept it.
- **Benefit:** If you decided to switch from Google Drive to Dropbox, you would only need to update the logic in this specific layer.

5. Backup Layer (The "Safety Net")

This is a specialized layer in your architecture designed for **Redundancy and Reliability**.

- **Responsibility:** It monitors the Data Layer and ensures that information isn't just sitting in one place. It triggers the scripts that copy local data to the cloud.
- **Failover Logic:** It likely contains the "heartbeat" or "failover" logic you've been working on—ensuring that if the primary storage fails, the system knows exactly where the secondary copy is located.

Summary Table: How a Request Flows

Layer	Analogy	Component Example
Presentation	The Waiter	Django Templates (HTML)
Application	The Chef	Django Views / Logic
Data	The Pantry	Django Models / ORM
Integration	The Delivery Driver	Google Drive API Client
Backup	The Insurance Policy	Local-to-Cloud Backup Script

3.System

1. Stakeholder Roles & Workflow

The system acts as a bridge between the community and the government. Each role has a specific "clearance level" and set of tools.

User (The Citizen)

- **The Entry Point:** Everything starts here. A user doesn't just "post" a complaint; they create a **documented record**.
- **Self-Service Tracking:** Instead of calling an office, the user accesses a personalized dashboard. This reduces the administrative load on the city.
- **Interaction:** They provide the raw data (images, descriptions, location) that the rest of the system relies on.

Admin (The Orchestrator)

- **Traffic Control:** The Admin doesn't necessarily fix the problems; they ensure the right problem gets to the right person. They use a high-level dashboard to view "bottlenecks" (e.g., if one authority has 50 pending tasks).
- **User Management:** They handle security, such as resetting passwords or blocking malicious accounts, ensuring the platform remains trusted.

Authority (The Resolver)

- **Field Execution:** These are the departmental heads or workers (e.g., Water Dept, Electricity Board).
- **Accountability:** By updating the status (e.g., "In Progress" to "Resolved"), they provide the digital "proof" of work that the User sees on their end.

2. The Data Safety Mechanism (The "Insurance")

Since this is a **Smart City** initiative, data loss isn't just a technical glitch—it's a loss of civic record. Your architecture addresses this through a **Hybrid Backup Strategy**.

Automated Local Backups

- **The First Line of Defense:** At scheduled intervals (e.g., every 24 hours), the system triggers a script that "dumps" the SQL database and user-uploaded media into a local server directory.
- **Speed:** Local backups are the fastest way to recover if a small error occurs (like an accidental deletion).

Cloud Integration (The Off-site Vault)

- **Disaster Recovery:** Using the **Integration Layer**, the system pushes these local files to **Google Drive**.
- **The "Failover" Logic:** If the physical local server crashes (hardware failure), the Admin can "pull" the latest backup from the cloud, ensuring the system is back online with minimal **RTO (Recovery Time Objective)**.

3. System Interaction Logic

The system interaction logic defines how a complaint flows through different layers of the application, ensuring proper handling, processing, and secure storage of data. The system follows a layered architecture where each layer performs a specific function, resulting in an efficient and reliable workflow.

Submission: The process begins when a user submits a complaint through the Presentation Layer. This is done using the web interface, where the user provides necessary details such as title, description, and any supporting images or documents. The interface is designed to be user-friendly, ensuring that users can easily register their complaints without complexity.

Validation: Once the complaint is submitted, the Application Layer receives the data and performs validation checks. These checks ensure that all required fields are filled correctly and that the data is in the proper format. If any errors are found, the user is prompted to correct them. After successful validation, the data is stored in the Data Layer (database) using Django ORM, ensuring structured and secure storage.

Assignment: After the complaint is stored, the administrator logs into the system and views all pending requests. The Application Layer retrieves these requests from the database and presents them to the admin. The administrator then assigns each complaint to the appropriate authority based on the type of issue. This assignment is recorded in the system, establishing a link between the request and the authority.

Action: The assigned authority logs into their dashboard and views the complaints assigned to them. They take necessary action to resolve the issue and update the status of the complaint (such as ongoing or completed). These updates are immediately reflected in the system, allowing users and administrators to track progress in real time.

Protection: While all these operations are taking place, the Backup Layer runs in the background. It automatically triggers backup processes whenever changes occur in the system. The data is copied and stored in both local storage and cloud storage (Google Drive). This ensures redundancy, meaning that even if one storage location fails, the data can be recovered from another source.

Overall, this interaction logic ensures a smooth flow of information between users, administrators, and authorities while maintaining data integrity, system reliability, and continuous protection through automated backups.

2.6 Requirements

The requirements of this project were defined to ensure that the system is functionally complete, user-friendly, and executable in a practical development environment.

2.6.1 Functional requirements

The functional requirements define the core functionalities of the system.

- User registration and login
- Complaint submission with details and images
- Admin assignment of complaints
- Authority status updates
- Feedback submission by users
- Automated backup triggering
- Backup storage in local and cloud
- Data restore functionality

These features ensure that the system performs all required operations efficiently.

2.6.2 User requirements

The system is designed to meet the needs of different users:

- Easy-to-use interface
- Secure login system
- Ability to track complaint status
- Fast response from authorities
- Reliable data storage

The system ensures a smooth user experience and high satisfaction.

2.6.3 Environmental requirements

The system requires a stable environment for proper functioning:

• Operating System: Windows/Linux
• Software: Python, Django, MySQL
• Internet connection for cloud backup
• Web browser for accessing the system

These requirements ensure smooth operation of the application.

3.2 Design and Architecture

1. The Modular Ecosystem

Instead of one giant block of code, your project is divided into specialized "compartments." This is essential for your 18-page technical report because it proves the system is **maintainable**.

User Interface (UI) Module

- **What it is:** The frontend dashboard built with Django Templates.
- **Project Role:** This is where the citizen fills out the grievance form (Location, Category, Description) and where the Admin "drags and drops" a complaint to an Authority.
- **Benefit:** If you want to change the look of the "edustream" or "Smart City" portal to a dark mode, you only edit the CSS/HTML here. The logic remains untouched.

Application Logic Module

- **What it is:** The "engine room" (Django Views).
- **Project Role:** This handles the **Status Lifecycle**. It dictates that a complaint can only move from *Pending → Assigned → Resolved*. It also contains the "if/else" logic that checks if a local backup was successful before attempting a cloud upload.

Database Module

- **What it is:** The persistent memory (Django Models/ORM).
- **Project Role:** It stores the relationships. For example, it links a `ComplaintID` to a specific `UserID` and a specific `AuthorityID`.
- **Benefit:** Because Django handles the database module, you are protected against data corruption during high-traffic periods in a "Smart City" scenario.

Backup Module (The Failover Layer)

- **What it is:** The integration between your local server and the Google Drive API.
- **Project Role:** This is your **Redundancy Engine**. It independently pulls data from the Database Module and mirrors it to the cloud. It ensures that even if the physical local server is destroyed, the citizen's grievance records are safe.

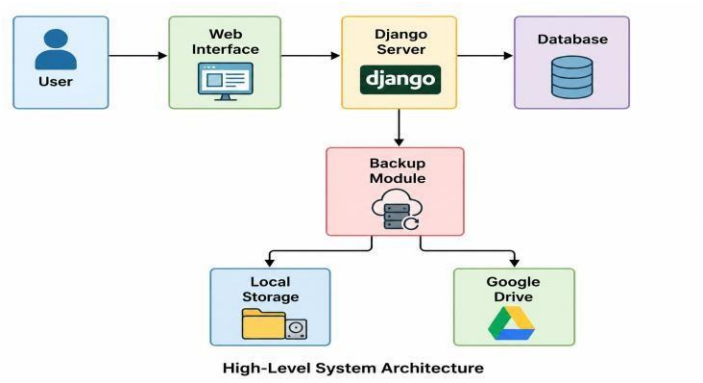
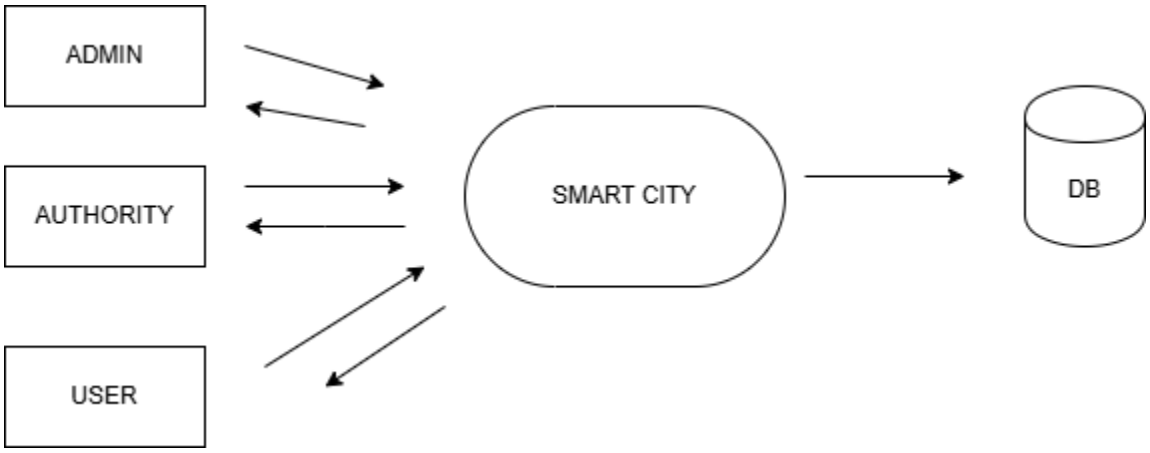


Figure 1: High-Level System Architecture of the Proposed System

The above diagram represents the overall architecture of the system. It shows how user requests are processed through the web interface and handled by the Django

server. The backup module ensures that data is stored both locally and on cloud storage, providing redundancy and reliability.

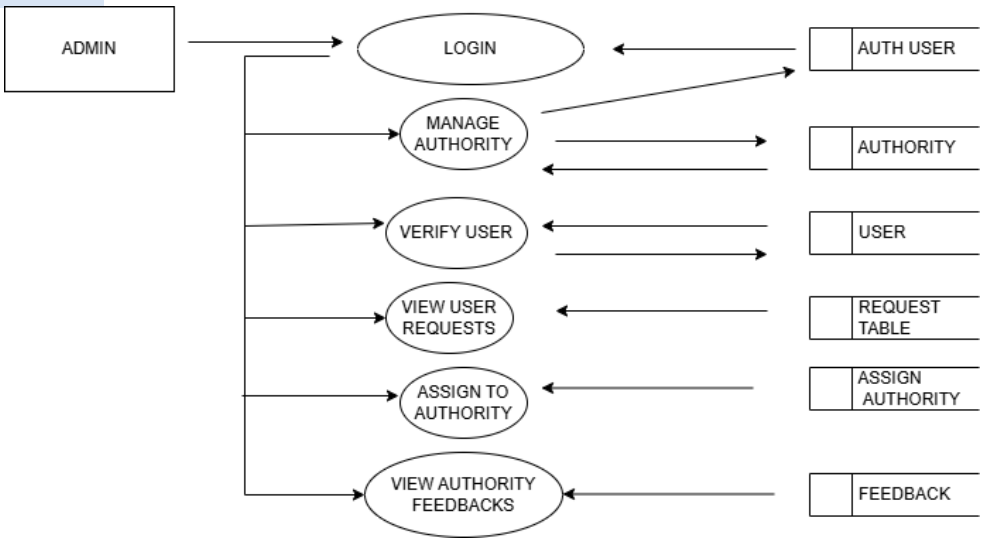
DFD LEVEL 0



The above diagram represents the Level 0 Data Flow Diagram (DFD) of the Smart City system. It shows the interaction between three main external entities: User, Admin, and Authority, with the central system.

The User submits complaints and views updates through the system. The Admin manages the system by assigning complaints to the appropriate authorities. The Authority receives assigned tasks, performs the required actions, and updates the status.

All the data generated through these interactions is stored in the database (DB). This diagram provides a high-level overview of how data flows between users and the system, ensuring proper communication and data management.



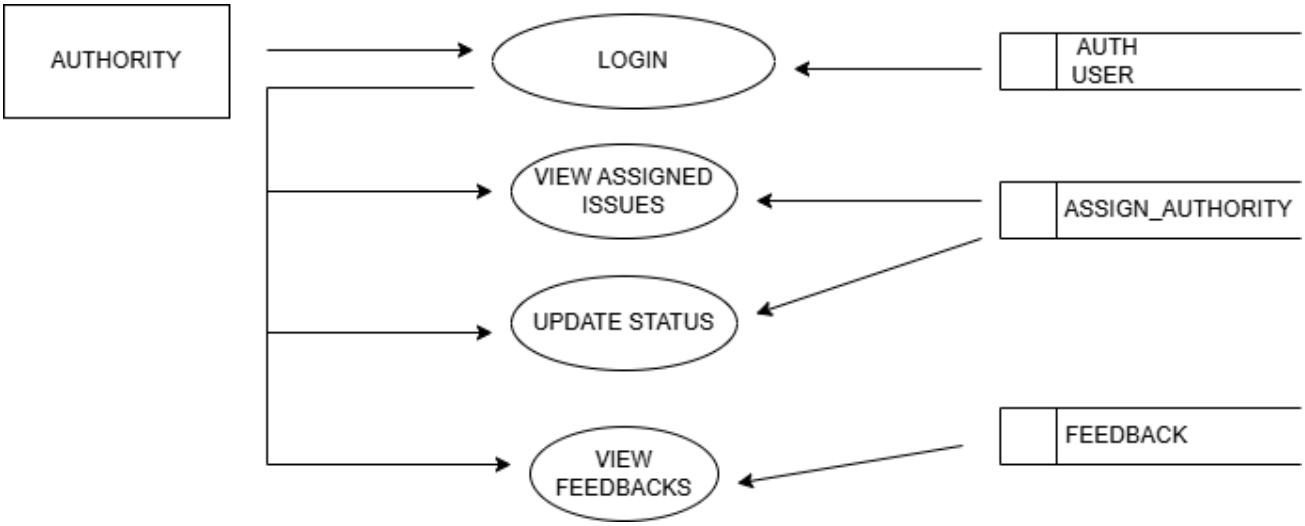
The above diagram represents the Level 1 Data Flow Diagram (DFD) of the Smart City system. It provides a detailed view of the internal processes handled by the system, particularly from the administrator's perspective.

The Admin interacts with the system through various processes such as login, managing authorities, verifying users, viewing user requests, assigning tasks to authorities, and viewing feedback. Each of these processes communicates with different data stores such as user data, request table, assignment records, and feedback.

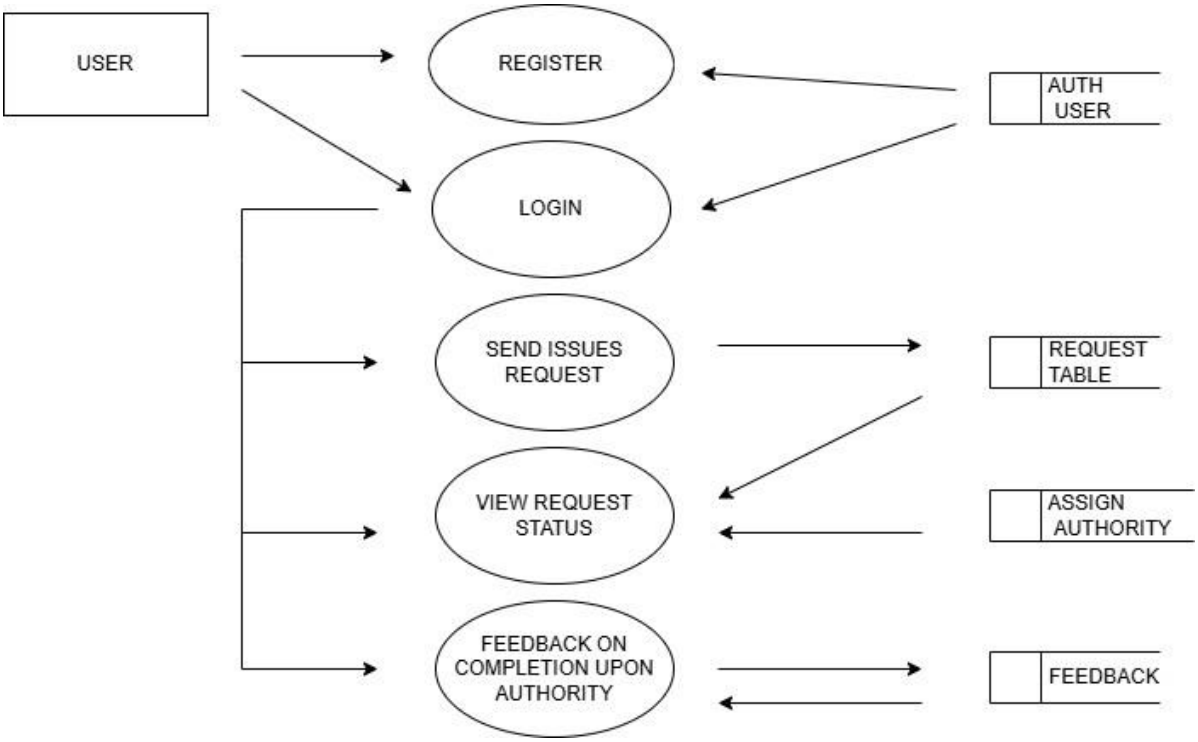
The login process authenticates users, while the manage authority and verify user processes handle system administration tasks. The admin can view all user requests and assign them to appropriate authorities. Authorities then update the status, and users can provide feedback, which is stored in the feedback module.

This diagram shows how data flows between processes and data stores, ensuring proper management and coordination within the system.

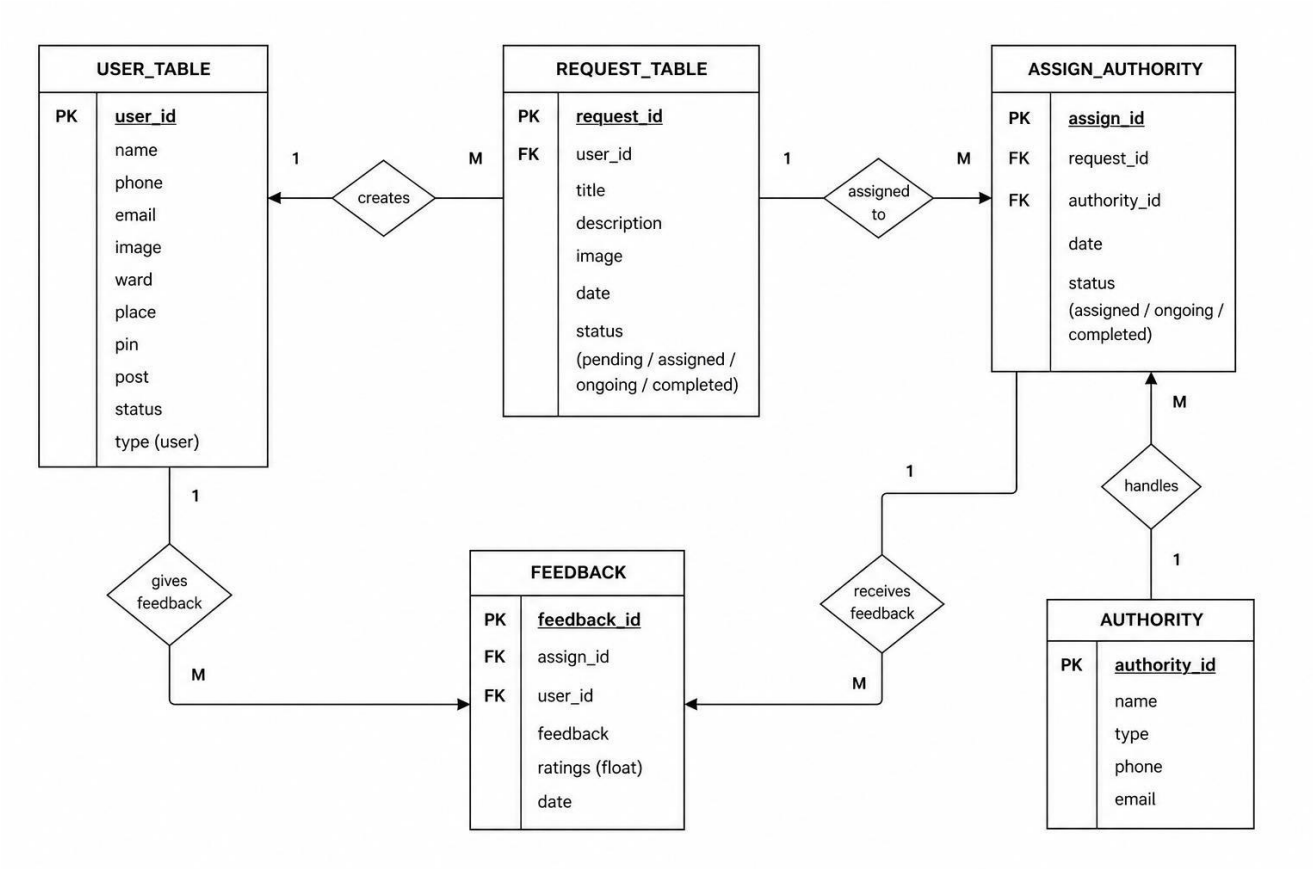
DFD LEVEL 2



DFD LEVEL 3



ER DIAGRAM



The ER diagram represents the structure of the system database and the relationships between different entities. A user can create multiple requests, and each request is assigned to an authority through the assign_authority entity. Authorities are responsible for handling and updating the status of these requests. After completion, users can provide feedback, which is linked to both the assignment and the user. This design ensures proper data organization and efficient management of the system workflow.

3.3 Implementation

The implementation of the system is carried out using the Django framework, which provides a structured approach to web development.

Models.py

Models define the structure of the database. In this system, models are used to represent entities such as users, authorities, requests, assignments, and feedback.

The system uses Django models to define database structure. For example, the user_table model stores user details such as name, email, and phone number.

Code snippet:

```
class authority(models.Model):
    LOGIN=models.ForeignKey(User,on_delete=models.CASCADE)
    name=models.CharField(max_length=50)
    type=models.CharField(max_length=100)
    phone=models.BigIntegerField()
```

Views:

Views handle the business logic of the application. They process user requests, interact with the database, and return appropriate responses.

```
def _upload_drive(filepath):
    try:
        drive = _get_drive()
        drive_file = drive.CreateFile({"title": os.path.basename(filepath)})
        drive_file.SetContentFile(filepath)
        drive_file.Upload()
        log.info(f"[DRIVE  OK] Uploaded: {os.path.basename(filepath)}")
        return True
    except Exception as exc:
        log.warning(f"[DRIVE  FAIL] Upload failed: {exc}")
        return False
```

URLs:
The URL configuration maps user requests to specific views. It defines the routing of the application.

```
path('login_get/',views.login_get),
path('admin_addauthority/',views.admin_addauthority),
path('admin_assignauthority/<id>',views.admin_assignauthority),
path('admin_authorityfeedback/',views.admin_authorityfeedback),
```

signals

Django signals are used to trigger backup operations automatically. Whenever a request is created or updated, the backup process is initiated.

```
@receiver(post_save, sender="myapp.request_table")
def on_request_saved(sender, instance, created, **kwargs):
    """
    Fires when a user request is created (submitted) or its status is updated
    (pending → assigned → ongoing → completed). Both are worth capturing immediately.
    """
    action = "submitted" if created else f"updated to '{instance.status}'"
    print(f"[SIGNAL] Request {instance.id} {action} → backup queued")
    trigger_backup_async()
```

Backup System:

The backup system is implemented using Python scripts. It performs the following operations:

- Creates database dump
- Compresses media files

• Stores backup locally
• Uploads backup to Google Drive

```
def run_full_backup():
    issues = _validate_required_paths()

    log.info("=" * 50)
    log.info("[BACKUP] Full backup started")

    if issues:
        for issue in issues:
            log.error(f"[CONFIG FAIL] {issue}")
        log.info("[BACKUP] Full backup stopped")
        log.info("=" * 50)
        return False
```

MEDIA BACKUP FUNCTION

```
def backup_media():
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    base_path = os.path.join(BACKUP_MEDIA_DIR, f"media_{timestamp}")
    zip_path = base_path + ".zip"

    log.info("[MEDIA] Zipping media directory...")
    shutil.make_archive(base_path, "zip", MEDIA_DIR)
```

This function creates a compressed ZIP backup of all media files. This includes images and uploaded files. Compression reduces storage space and allows easy transfer to cloud storage.

DATABASE BACKUP FUNCTION:

This function creates a backup of the MySQL database using the mysqldump command. The backup file is stored locally with a timestamp to ensure uniqueness. This ensures that database data can be restored in case of failure.

```python
def backup_database():
    filename = f"db_{datetime.now().strftime('%Y%m%d_%H%M%S')}.sql"
    filepath = os.path.join(BACKUP_DB_DIR, filename)

    log.info("[DB] Running mysqldump...")
    try:
        with open(filepath, "w", encoding="utf-8") as out_file:
            result = subprocess.run(
                ["mysqldump", f"--defaults-extra-file={MYSQL_CNF}", DB_NAME],
                stdout=out_file,
                stderr=subprocess.PIPE,
                text=True,
                timeout=60,
            )
```

Restore System:

The restore system retrieves backup data and restores the database and media files.

The implementation ensures that all components work together seamlessly to provide a reliable system.

```python
def download_latest_from_drive():
    try:
        log.info("[DRIVE] Connecting to Google Drive...")
        drive = _get_drive()
        files = drive.ListFile({"q": "trashed=false"}).GetList()
        db_files = [
            item for item in files
            if item["title"].startswith("db_") and item["title"].endswith(".sql")
        ]
```

3.4 Testing

TESTING

Testing was performed through systematic manual scenarios and functional validation checks to ensure the correctness, reliability, and performance of the system. The testing process covered all major modules including user operations, admin controls, authority actions, and the backup system.

Functional Test Scenarios:

- User registration with valid details succeeds, and invalid inputs are rejected with appropriate error messages.
- User login with correct credentials is successful, while incorrect credentials display an error message.
- Complaint submission with valid data is stored successfully in the database.
- Complaint submission with missing or invalid fields triggers validation errors.
- Admin can view all user complaints without any issues.
- Admin can assign complaints to the appropriate authority successfully.
- Authorities can view assigned complaints and update their status correctly.

- Status updates (pending, ongoing, completed) are reflected accurately across the system.
- Users can submit feedback after the completion of a complaint.
- Backup is automatically triggered when a complaint is created or updated.
- Database backup is generated successfully and stored in local storage.
- Media files are compressed and backed up correctly.
- Backup files are uploaded to cloud storage (Google Drive) successfully.
- In case of cloud failure, the backup is safely retained in local storage.
- Restore functionality retrieves and restores data correctly from backup files.

Data Consistency Tests:

- Complaint status updates are consistently reflected across all modules.
- Assigned complaints correctly maintain relationships between user, authority, and request without data mismatch.
- Repeated updates modify existing records instead of creating duplicate entries.

- Feedback is correctly linked to the respective user and assigned request.
- Database records remain consistent after page refresh or repeated access.
- Backup files always contain the most recent updated data.
- Restored data matches the original data without any loss or corruption.
-

Usability Tests:

- All system features such as login, complaint submission, dashboards, and feedback are easily accessible.
- Buttons, forms, and navigation elements are clearly visible and user-friendly.
- System messages such as success alerts, errors, and status updates are understandable.
- Navigation between user, admin, and authority panels is smooth and intuitive.
- Forms include proper validation messages to guide the user.
- The interface is simple and easy to use, even for users with minimal technical knowledge.

System Stability Test:

The system remains stable during continuous operations such as multiple complaint submissions, status updates, and dashboard access. The performance does not degrade, and the interface remains responsive and readable.

Known Testing Limitations:

- No automated testing framework has been implemented; testing was conducted manually.
- High-concurrency testing was not performed, as the system was tested with a limited number of users.
- Cross-browser testing was limited, and no automated scripts were used for browser compatibility verification.
- Cloud backup testing depends on internet connectivity, which may affect real-time validation in some cases.
- Overall, the testing results confirm that the system functions correctly, maintains data integrity, and provides reliable backup and recovery mechanisms.

3.5 Graphical User Interface (GUI) Layout

The Graphical User Interface (GUI) of the Smart City Issue Management System is designed to provide a user-friendly, efficient, and visually structured environment for all users, including Admin, Authority, and General Users. The interface follows a clean and organized dashboard-based layout with a clear visual hierarchy to ensure ease of navigation and usability.

The GUI is developed using HTML, CSS, and Django templates, ensuring dynamic content rendering and smooth interaction between the frontend and backend.

Top Navigation:

The top navigation bar serves as the primary access point for users to move between different sections of the application. It includes links such as Home, Login, Register, Dashboard, and Logout. This ensures that users can quickly navigate without confusion.

Code Snippet:

```
<nav class="nav-grid">
    <a href="/myapp/admin_home/">Home</a>
    <a href="/myapp/admin_viewauthority/">Manage Authority</a>
    <a href="/myapp/admin_verify/">Verify Users</a>
    <a href="/myapp/admin_viewreq/">User Requests</a>
    <a href="/myapp/admin_view_assign/">Assignments</a>
    <a href="/myapp/admin_authorityfeedback/">Authority Feedbacks</a>
</nav>
```

The navigation bar is designed to remain consistent across all pages, providing a uniform user experience.

Home / Landing Page:

The home page acts as the entry point of the system. It provides a brief introduction to the application and highlights its purpose. It includes primary action buttons that guide users to login or register.

Code Snippet:

```
<body>
    <div class="dashboard-shell">
        <header class="topbar">
            <div class="topbar-head">
                <div class="brand">
                    <h1>Admin Dashboard</h1>
                    <p>Monitor users, authorities, requests, assignments, and complaint feedback fro
                </div>
                <a href="/myapp/login_get/" class="logout-link">Log Out</a>
            </div>
```

This section ensures that users immediately understand the purpose of the system and can proceed with the next steps.

User Dashboard:

The user dashboard is designed to allow users to interact with the system easily. Users can submit complaints, view their complaint status, and provide feedback.

Code Snippet:

```
<div class="dashboard-shell">
    <header class="topbar">
        <div class="brand">
            <h1>Citizen Dashboard</h1>
            <p>Submit complaints, track progress, and review the authority response in one place.
        </div>
        <div class="topbar-actions">
            <button t  Follow link (ctrl + click)  age-btn soft" onclick="window.history.back()">Back</but
            <a href="/myapp/login_get/" class="page-btn primary">Log Out</a>
        </div>
```

The layout is simple and ensures that users can access key features without difficulty.

Admin Dashboard:

The admin dashboard provides full control over the system. It allows the admin to manage users, view complaints, assign authorities, and monitor system activities.

Code Snippet:

The admin interface is designed for efficiency, allowing quick access to all administrative functions.

Authority Dashboard:

The authority dashboard enables authorities to view assigned complaints and update their status. This ensures timely resolution of issues.

Code Snippet:

```
<div class="dashboard-shell">
    <header class="topbar">
        <div class="brand">
            <h1>Authority Dashboard</h1>
            <p>Manage complaints, update progress, and review citizen feedback without layout co
        </div>
```

This section ensures that authorities can manage their tasks effectively.

Complaint Submission Form:

The complaint submission form is a key component of the system. It allows users to report issues by entering relevant details such as title, description, and uploading images.

Code Snippet:

```
<div class="form-container">
  <h2>Send Request</h2>

  <form method="post" enctype="multipart/form-data"> {% csrf_token %}

    <label for="title">Title:</label>
    <input type="text" id="title" name="title" placeholder="Enter issue title" required>

    <br><br>

    <label for="description">Description:</label>
    <input type="text" id="description" name="description" placeholder="Describe the issue" required>
```

</form>

Form validation ensures that users provide correct and complete information before submission.

Status Tracking Section:

Users can track the progress of their complaints through the status tracking section. This improves transparency and user satisfaction.

Code Snippet:

```
<div class="container">
    <h2>Your Complaints / Requests</h2>

    <table>
        <thead>
            <tr>
                <th>#</th>
                <th>Title</th>
                <th>Description</th>
                <th>Image</th>
                <th>Status</th>
                <th>Action</th>
            </tr>
```

The table format provides a clear and structured view of complaint statuses.

Feedback Section:

After the issue is resolved, users can provide feedback and ratings.

Code Snippet:

```
<tr>
    <td>{{ forloop.counter }}</td>
    <td>{{ i.feedback }}</td>
    <td class="rating">&#11088; {{ i.ratings }}/5</td>
    <td>{{ i.ASSIGN.REQUEST.title }}</td>
    <td>{{ i.USER.name }}</td>
    <td>{{ i.date }}</td>
</tr>
```

This helps in evaluating the performance of authorities and improving the system.

Overview Metrics:

The system displays important metrics such as total complaints, pending issues, and completed tasks. These metrics help administrators monitor system performance.

These indicators provide quick insights into system activity.

UI Design Principles Followed:

The GUI is designed based on key usability principles to enhance user experience:

Consistency:
All pages maintain uniform design, colors, and layout for familiarity.

Simplicity:
The interface avoids unnecessary complexity, making it easy to use.

Accessibility:
Readable fonts, proper spacing, and contrast improve visibility.

Responsiveness:
The layout adapts to different screen sizes and devices.

Feedback Mechanism:
Users receive immediate responses such as success messages and error alerts.

Overall, the GUI ensures smooth interaction between users and the system, making the application efficient, reliable, and easy to use.

3.6 **CUSTOMER TESTING**

Customer testing was conducted to evaluate the usability, functionality, and overall performance of the Smart City Issue Management System from an end-user perspective. The testing involved a small group of users interacting with the system to perform real-world tasks such as registration, complaint submission, status tracking, and feedback submission.

Users were first asked to register and log into the system. The registration and login processes were found to be simple and user-friendly, with clear instructions and validation messages. Most users were able to complete these steps without any assistance.

After logging in, users tested the complaint submission feature by entering issue details and uploading images. The system successfully stored the complaints and displayed confirmation messages. Users also verified that they could track the status of their complaints in real time.

The admin functionalities were tested by assigning complaints to authorities. Authorities then logged into their dashboards, viewed assigned complaints, and updated their status. The updates were immediately reflected in the user interface, confirming proper system synchronization.

The feedback module was also tested, where users provided ratings and comments after issue resolution. This feature worked correctly and helped in evaluating service quality.

Special attention was given to the backup system during customer testing. Users performed actions such as submitting complaints and updating statuses, after which backups were verified in both local storage and cloud storage (Google Drive). This confirmed that the backup mechanism was functioning reliably.

Overall, users found the system easy to navigate, efficient, and responsive. The

interface was intuitive, and all major functionalities worked as expected. Minor suggestions included improving visual design and adding more user guidance in certain sections.

Customer testing confirmed that the system meets user requirements and performs effectively in real-world scenarios.

3.7 EVALUATION

The project successfully meets its core objectives by implementing a complete Smart City Issue Management System with integrated backup functionality. The system enables users to submit complaints, allows administrators to assign them to appropriate authorities, and ensures that authorities can update the status efficiently.

The system demonstrates a complete workflow from complaint submission to resolution and feedback. The integration between frontend (user interface) and backend (Django server and database) is clearly implemented, ensuring smooth data flow and real-time updates. The backup system further enhances the project by providing data protection through both local storage and cloud storage (Google Drive).

The application maintains persistent data using a structured database, ensuring that all user activities, complaints, assignments, and feedback are stored and managed reliably. The automatic backup mechanism ensures data safety and recovery, which is a key feature of the system.

- Strengths:

The project follows a clear modular architecture, separating user, admin, authority, and backup functionalities.

The system provides a complete real-world workflow for issue management in a smart city environment.

Frontend and backend integration is effectively implemented using Django.

The backup system adds reliability by ensuring data redundancy through local and cloud storage.

The user interface is simple, intuitive, and easy to navigate.

The system provides real-time updates for complaint status and feedback.

- Limitations:

The system is tested in a controlled environment and may not handle high concurrency effectively.

Advanced authentication and authorization mechanisms are limited.

Security features such as data encryption and role-based access control can be further improved.

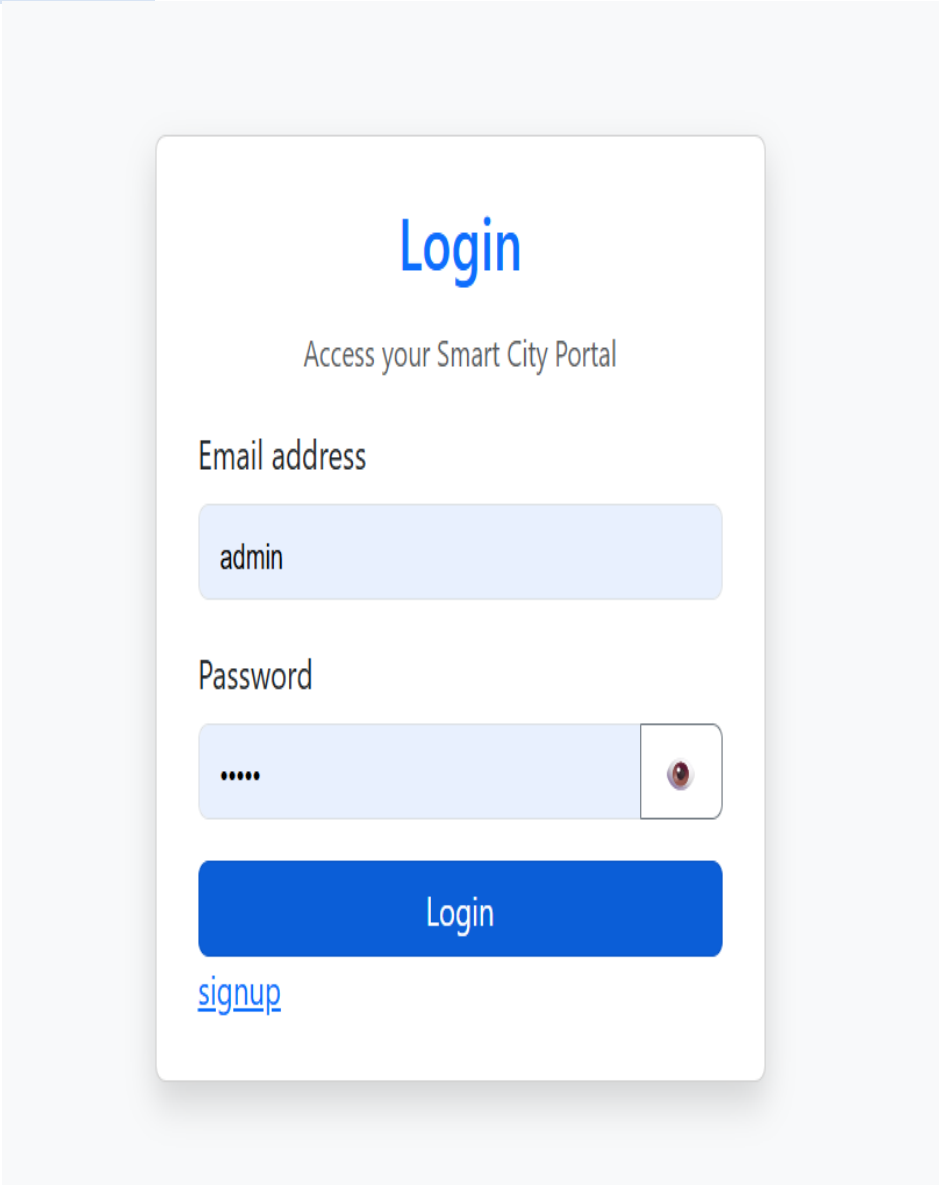
The backup system depends on internet connectivity for cloud storage operations.

Testing was primarily manual, and automated testing tools were not implemented.

Overall, the project demonstrates a functional, reliable, and practical solution for managing city-related issues while ensuring data safety through backup mechanisms.

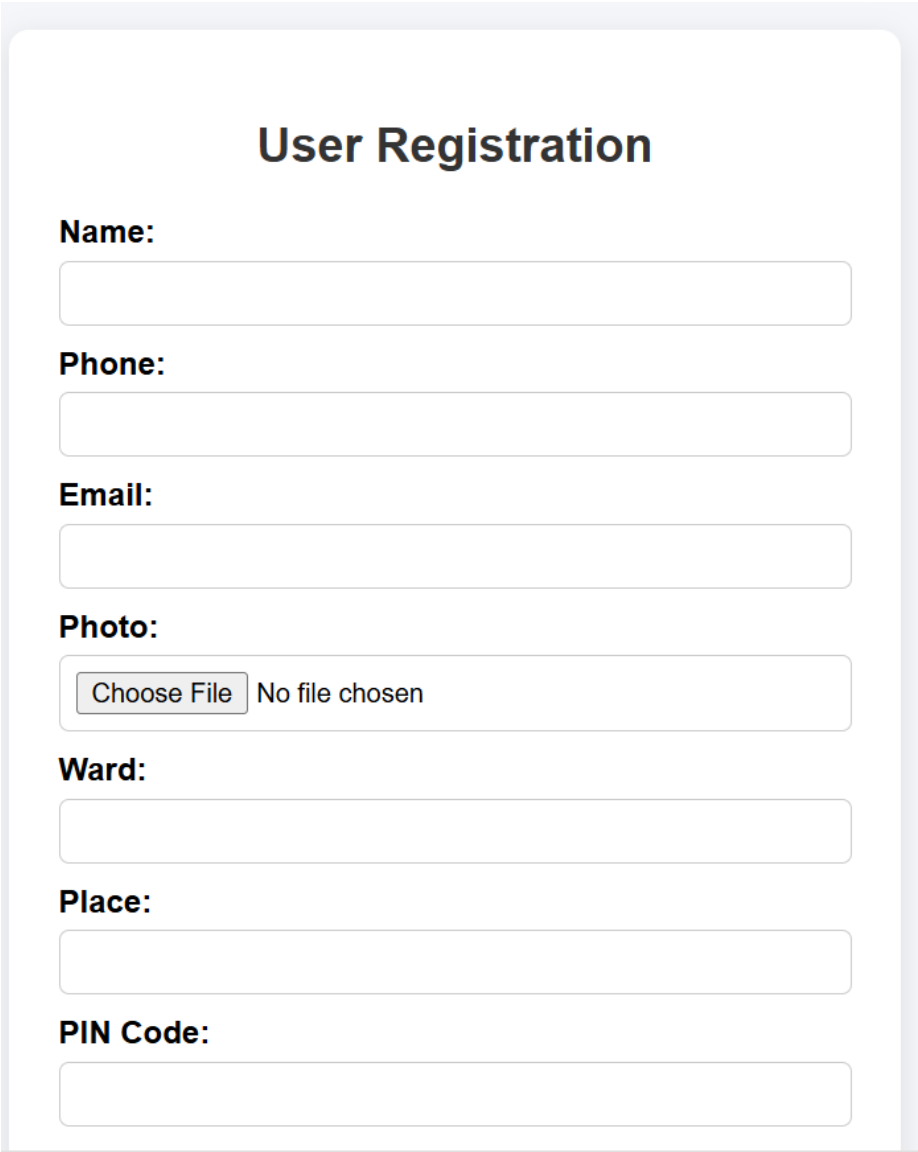
4. SNAPSHOTS

This section presents the visual snapshots of the developed system. The screenshots demonstrate the functionality of different modules including user interaction, admin control, authority actions, and backup operations. These images provide clear evidence of the system's working and user interface design.

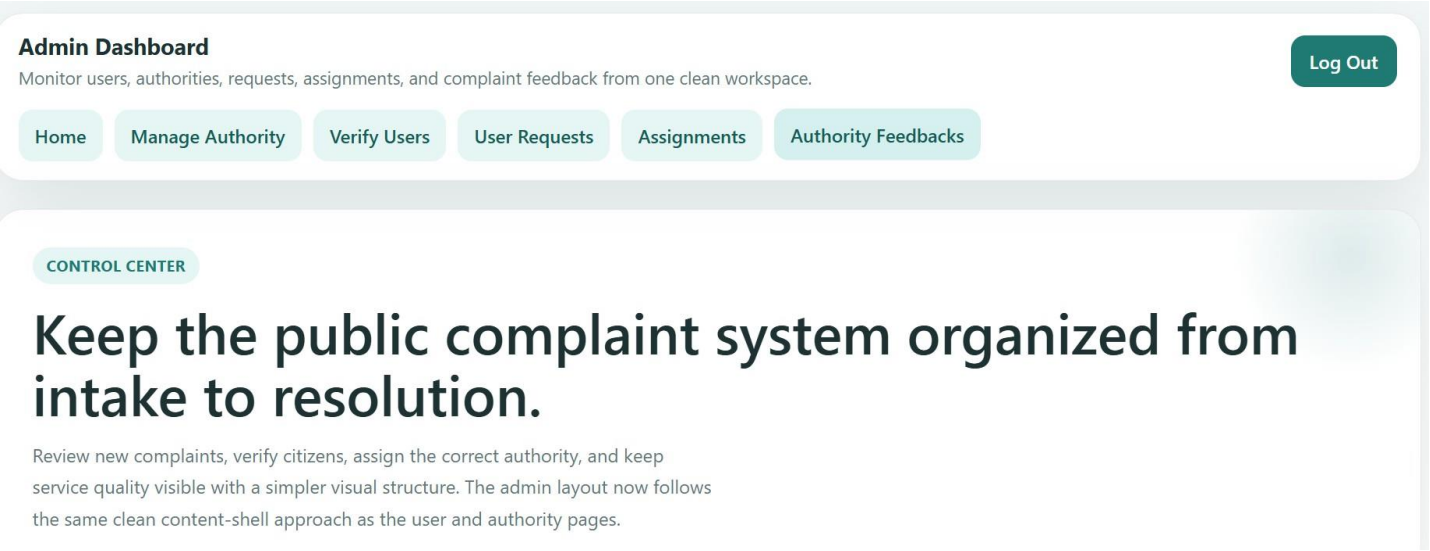

Login
Access your Smart City Portal
Email address
admin
Password
Login
signup

LOGIN

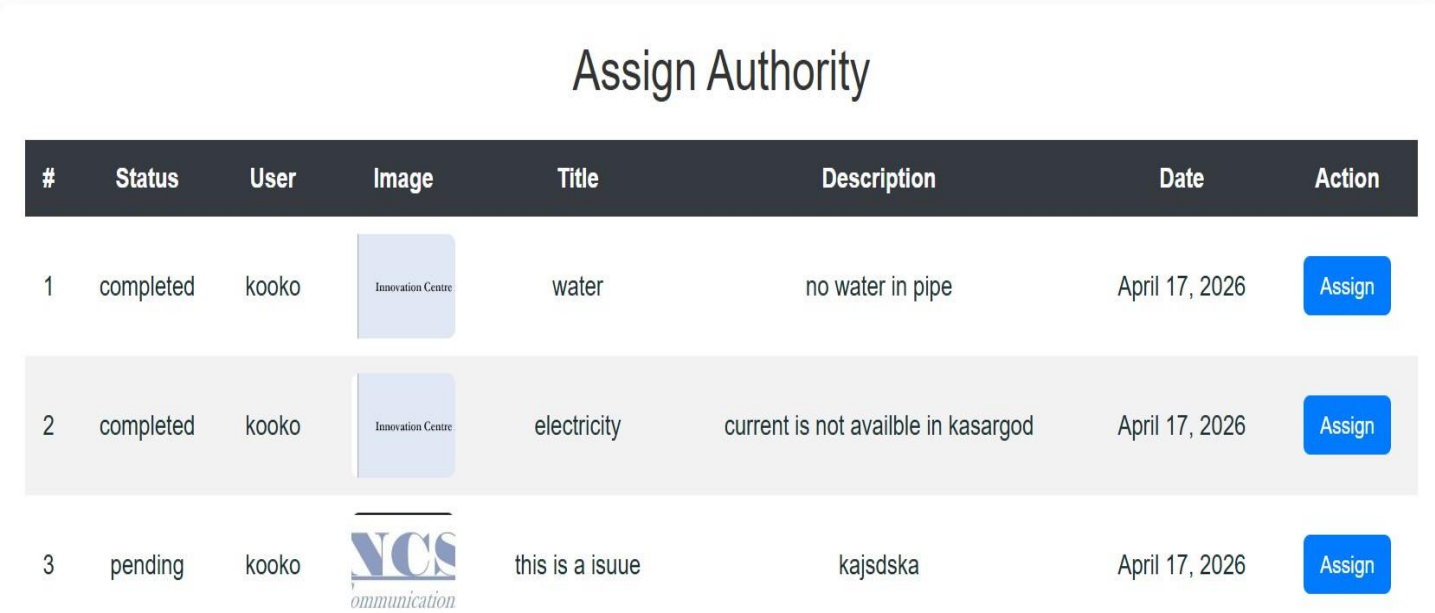
USER REGISTRATION PAGE



ADMIN DASHBOARD



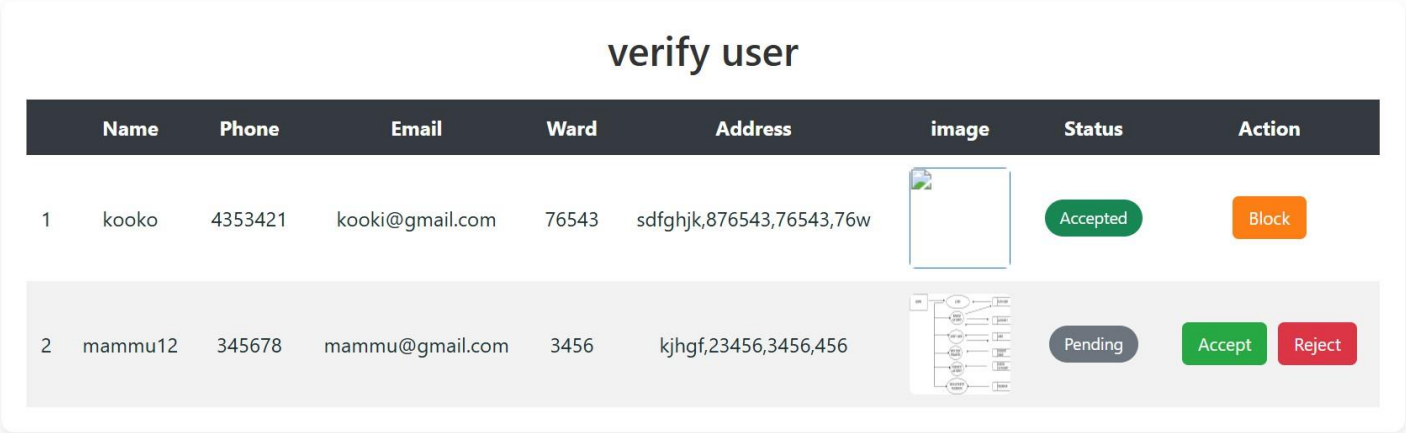
View complaints and assign



Assign Authority

#	Status	User	Image	Title	Description	Date	Action
1	completed	kooko	Innovation Centre	water	no water in pipe	April 17, 2026	Assign
2	completed	kooko	Innovation Centre	electricity	current is not availble in kasargod	April 17, 2026	Assign
3	pending	kooko	NCS ommunication	this is a isuue	kajsdska	April 17, 2026	Assign

User verification

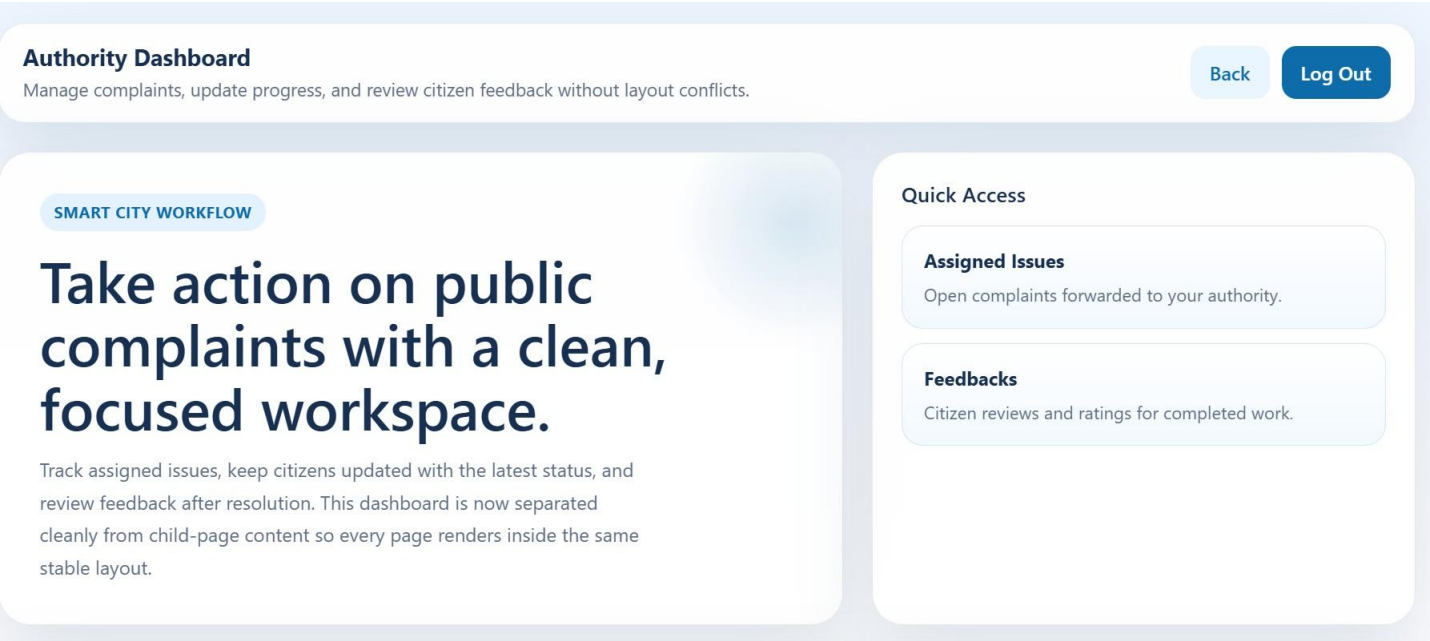


verify user

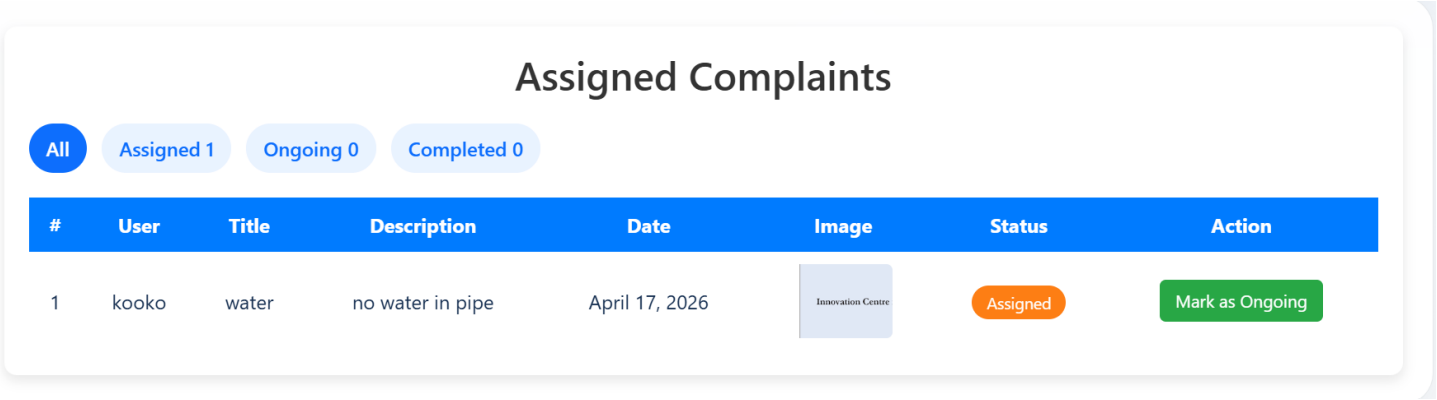
	Name	Phone	Email	Ward	Address	image	Status	Action
1	kooko	4353421	kooki@gmail.com	76543	sdfghjk,876543,76543,76w		Accepted	Block
2	mammu12	345678	mammu@gmail.com	3456	kjhgf,23456,3456,456		Pending	Accept Reject

AUTHORITY

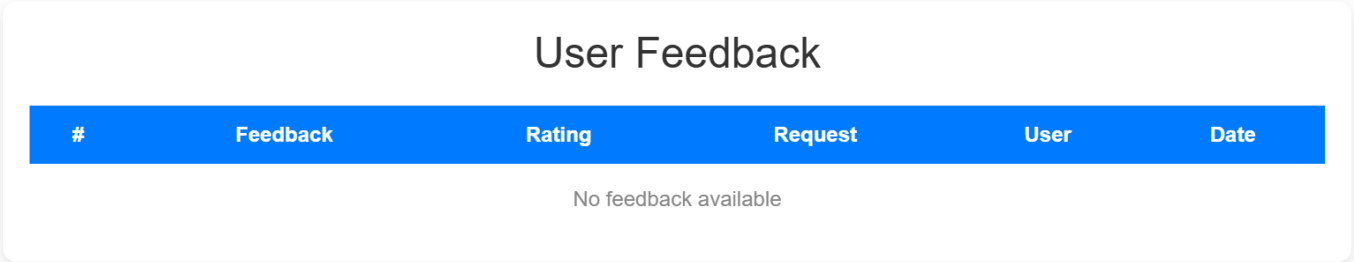
Authority dashboard



View assigned complaints



View user feedback

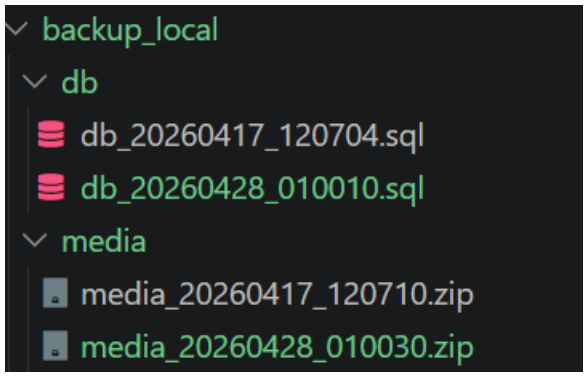


GOOGLE DRIVE UPLOAD SCREEN

Name	Details
media_20260428_010030.zip	You uploaded • 01:00
db_20260428_010010.sql	You created • 01:00

Google Drive Backup Storage

- **Backup folder (local storage)**



Local Backup Files

DATABASE

id bigint	image varchar(100)	descreption varchar(50)	date date	title varchar(50)	USER_id bigint	status varchar(5
1	data.jpg	hfhffyfy	2026-04-13	water	10	comple
2	data_TlgkCbI.jpg	khkhh	2026-04-13	water	10	send
3	data_d5DLB9A.jpg	wdwdwdwd	2026-04-13	dwwdwd	10	send
4	sslc_certificate_CDrcsFl.jpeg	sdsdsdsd	2026-04-13	ssasasasass	10	send
5	sslc_certificate_P2JzCgj.jpeg	hhhjh	2026-04-13	water	10	send
6	ibm.png	no water in pipe	2026-04-17	water	10	assigne

5. CONCLUSION

Conclusions

This project demonstrates the successful development of a web-based Smart City Issue Management System with an integrated backup mechanism. The system evolved from the concept of managing city-related complaints into a fully functional web application that supports real-time interaction between users, administrators, and authorities.

The project validates full-stack development concepts by integrating a structured frontend interface, a Django-based backend, and a persistent database layer into a unified system. It enables users to submit complaints, track their status, and provide feedback, while administrators efficiently manage and assign tasks to authorities. Authorities can update the progress of assigned tasks, ensuring transparency and accountability throughout the process.

A key highlight of the system is the implementation of an automated backup mechanism. The system ensures data safety by storing backups both locally and on cloud storage (Google Drive). This enhances reliability and provides a recovery solution in case of data loss or system failure.

The overall architecture of the system is modular and well-organized, which improves maintainability and scalability. Each component, including user management, request handling, assignment, feedback, and backup, is designed with clear functionality and separation of concerns.

From an academic perspective, the project successfully meets objectives related to system design, implementation, testing, user interface development, and evaluation. It demonstrates practical application of theoretical concepts and provides a solid foundation for future enhancements such as improved security, scalability, and mobile integration.

In conclusion, the project delivers a reliable, efficient, and user-friendly solution for managing city issues while ensuring data protection through an effective backup system.

6. FURTHER FUTURE DEVELOPMENTS

Further Development or Research

The current system provides a functional and reliable solution for managing city-related issues with an integrated backup mechanism. However, there are several areas where the system can be enhanced and extended to improve performance, scalability, and user experience.

Recommended Future Scope:

Enhance database scalability by optimizing queries and supporting larger datasets for real-time city-wide deployment.

Implement advanced authentication and role-based authorization to improve security and control access for users, admins, and authorities.

Develop a mobile application to provide easier access and real-time notifications for users and authorities.

Introduce real-time notifications (SMS/Email) to inform users about complaint status updates.

Improve the user interface with advanced design frameworks for better usability and responsiveness.

Extend the backup system by integrating additional cloud services such as AWS S3 for higher reliability and distributed storage.

Implement automated backup scheduling with more flexible configurations and monitoring features.

Add geolocation support to track and display complaint locations on a map for better visualization.

Develop an analytics dashboard for administrators to monitor complaint trends, resolution time, and system performance.

Introduce automated testing frameworks (unit testing, integration testing, and UI testing) to improve system reliability.

Optimize system performance using caching techniques and efficient data handling methods.

Enable multi-language support to make the system accessible to a wider range of users.

Research Directions:

Explore intelligent classification of complaints using machine learning techniques to automatically categorize issues.

Develop a smart recommendation system to assign complaints to the most suitable authority based on past performance and workload.

Implement real-time system updates using technologies such as WebSockets for instant status changes.

Enhance the backup system with predictive failure detection and automated recovery mechanisms.

Investigate the use of AI-based chat assistants to guide users in submitting complaints and navigating the system.

Study advanced security mechanisms such as data encryption and secure authentication protocols to protect sensitive user data.

Overall, these improvements and research directions can transform the current system into a more scalable, intelligent, and production-ready solution suitable for large-scale smart city applications.

7. REFERENCES

References

1. Django Documentation. (2025). Retrieved from https://docs.djangoproject.com/
Used for backend development, models, views, and URL routing.

2. Python Software Foundation. Python Documentation. Retrieved from https://docs.python.org/3/
Used for core programming concepts and standard libraries.

3. PyDrive2 Documentation. Retrieved from https://docs.iterative.ai/PyDrive2/
Used for integrating Google Drive cloud storage for backup.

4. MySQL Documentation. Retrieved from https://dev.mysql.com/doc/
Used for database design and management.

5. APScheduler Documentation. Retrieved from https://apscheduler.readthedocs.io/
Used for scheduling automated backup tasks.

6. HTML & CSS Documentation (MDN Web Docs). Retrieved from https://developer.mozilla.org/
Used for designing the user interface and frontend layout.

7. W3Schools Online Web Tutorials. Retrieved from https://www.w3schools.com/
Used for basic reference on HTML, CSS, and web development concepts.

8. GeeksforGeeks. Retrieved from https://www.geeksforgeeks.org/
Used for understanding Python, Django concepts, and problem-solving.

9. Stack Overflow. Retrieved from https://stackoverflow.com/
Used for debugging and resolving coding issues during development.

10. Google Drive API Documentation. Retrieved from https://developers.google.com/drive
Used for understanding cloud storage integration and file uploads.

11. Pressman, R. S. (2010). Software Engineering: A Practitioner's Approach.
Used for software development lifecycle and design concepts.

12. Sommerville, I. (2016). Software Engineering (10th Edition).
Used for system design, testing, and evaluation methodologies.

13. GitHub repositories and online code references used during development.

8. APPENDIX

Appendix

The appendix section contains supplementary materials that support the development and implementation of the Smart City Issue Management System. This includes source code, configuration details, and additional technical information that are not included in the main sections of the report.

Appendix A: Source Code

The complete source code of the project is provided in this section for reference. It includes all major modules used in the system:

- models.py – Defines the database structure including user, authority, request, assignment, and feedback entities.
- views.py – Contains the application logic for handling user requests, authentication, complaint submission, assignment, and status updates.
- urls.py – Defines URL routing and connects user requests to appropriate view functions.
- backup.py – Implements the automated backup system, including database backup, media backup, and cloud upload using Google Drive.
- restore.py – Handles restoration of backup data from local or cloud storage.
- signals.py – Triggers automatic backup when key operations such as request creation or update occur.

Sample Code Snippet:

Models.py

```
from django.db import models
from django.contrib.auth.models import User

# Create your models here.


class authority(models.Model):
    LOGIN=models.ForeignKey(User,on_delete=models.CASCADE)
    name=models.CharField(max_length=50)
    type=models.CharField(max_length=100)
    phone=models.BigIntegerField()
    email=models.CharField(max_length=50)

    def __str__(self):
        return self.name

class user_table(models.Model):
    LOGIN=models.ForeignKey(User,on_delete=models.CASCADE)
    name=models.CharField(max_length=50)
    phone=models.BigIntegerField()
    email=models.CharField(max_length=50)
    image=models.FileField()
    ward=models.IntegerField()
    place=models.CharField(max_length=100)
    pin=models.IntegerField()
    post=models.CharField(max_length=50)
    status=models.CharField(max_length=50)
    type=models.CharField(max_length=50, default='user')

    def __str__(self):
        return self.name


class request_table(models.Model):
    USER=models.ForeignKey(user_table,on_delete=models.CASCADE)
    image=models.FileField()
    descreption=models.CharField(max_length=50)
    date=models.DateField()
    title=models.CharField(max_length=50)
    status=models.CharField(max_length=50)

    def __str__(self):
```

```
        return self.title


class assign_authority(models.Model):
    REQUEST=models.ForeignKey(request_table,on_delete=models.CASCADE)
    AUTHORITY=models.ForeignKey(authority,on_delete=models.CASCADE)
    date=models.DateField()
    status=models.CharField(max_length=50)

    def __str__(self):
        return f"{self.REQUEST.title} - {self.AUTHORITY.name}"
class feedback(models.Model):
    ASSIGN=models.ForeignKey(assign_authority,on_delete=models.CASCADE)
    USER=models.ForeignKey(user_table,on_delete=models.CASCADE)
    feedback=models.CharField(max_length=50)
    ratings=models.FloatField()
    date=models.DateField()

    def __str__(self):
        return self.feedback
```

Backup.py

CORE BACKUP EXECUTION:

```
def run_full_backup():
    issues = _validate_required_paths()

    log.info("=" * 50)
    log.info("[BACKUP] Full backup started")

    if issues:
        for issue in issues:
```

```
        log.error(f"[CONFIG FAIL] {issue}")
    log.info("[BACKUP] Full backup stopped")
    log.info("=" * 50)
    return False

db_ok = backup_database() is not None
media_ok = backup_media() is not None

if db_ok or media_ok:
    log.info("[BACKUP] Full backup complete")
else:
    log.error("[BACKUP FAIL] No backup component completed successfully")

log.info("=" * 50)
return db_ok or media_ok
```

Signals.py

```
from django.db.models.signals import post_save, post_delete
from django.dispatch import receiver

from .backup import trigger_backup_async



@receiver(post_save, sender="myapp.request_table")
def on_request_saved(sender, instance, created, **kwargs):
    """
    Fires when a user request is created (submitted) or its status is updated
    (pending → assigned → ongoing → completed). Both are worth capturing immediately.
    """
    action = "submitted" if created else f"updated to '{instance.status}'"
    print(f"[SIGNAL] Request {instance.id} {action} → backup queued")
    trigger_backup_async()
```

25-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 1

For week commencing 25 March 2026

Programme: BCA (AI, Cloud Computing with DevOps)

Student Name: Muhammed Abdul Gafoor Register Number: **23BCAICD068**

WPPR: **1**

Internal Guide's Name:

MAJOR PROJECT Title:

Multi-Cloud Hybrid Strategy for Smart City

Targets set for the week:

- Finalize project idea and problem statement
- Study existing complaint management systems
- Understand Django framework basics
- Plan system modules and architecture

Progress/Achievements for the week:

- Selected project topic and defined objectives
- Completed requirement analysis
- Studied Django MVT architecture
- Designed initial system flow (User -> Admin -> Authority)
- Identified key modules (User, Request, Assignment, Backup)

1

25-03-2026

Future Work Plans:

- Start database design
- Create Django project structure
- Implement user authentication system

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

2

01-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 2

For week commencing 01 April 2026

Programme: BCA (AI, Cloud Computing with DevOps)

Student Name: Muhammed Abdul Gafoor Register Number: **23BCAICD068**

WPPR: **2**

Internal Guide's Name:

MAJOR PROJECT Title:

Multi-Cloud Hybrid Strategy for Smart City

Targets set for the week:

- Design database schema
- Create Django models
- Implement user registration and login

Progress/Achievements for the week:

- Designed database tables (user, authority, request, assignment, feedback)
- Implemented models in Django
- Established relationships using Foreign Keys
- Developed user registration system
- Implemented login functionality with role-based access

1

01-04-2026

Future Work Plans:

- Implement complaint submission system
- Develop admin module

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

2

08-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 3

For week commencing 08 April 2026

Programme: BCA (AI, Cloud Computing with DevOps)

Student Name: Muhammed Abdul Gafoor Register Number: **23BCAICD068**

WPPR: **3**

Internal Guide's Name:

MAJOR PROJECT Title:

Multi-Cloud Hybrid Strategy for Smart City

Targets set for the week:

- Implement complaint/request module
- Develop admin functionalities
- Implement assignment module

Progress/Achievements for the week:

- Developed complaint submission feature (user_sendreq)
- Implemented admin dashboard
- Added authority management (add, edit, delete)
- Implemented complaint assignment system
- Enabled status tracking (pending, assigned, ongoing, completed)

1

08-04-2026

Future Work Plans:

- Develop authority module
- Implement feedback system

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

2

15-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 4

For week commencing 15 April 2026

Programme: BCA (AI, Cloud Computing with DevOps)

Student Name: Muhammed Abdul Gafoor Register Number: **23BCAICD068**

WPPR: **4**

Internal Guide's Name:

MAJOR PROJECT Title:

Multi-Cloud Hybrid Strategy for Smart City

Targets set for the week:

- Develop authority module
- Implement feedback system
- Start backup system

Progress/Achievements for the week:

- Implemented authority dashboard
- Added status update features for authorities
- Developed feedback system with ratings
- Designed backup system architecture
- Started implementing database backup and media backup

1

15-04-2026

Future Work Plans:

- Complete backup and restore system
- Integrate cloud storage

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

2

22-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 5

For week commencing 22 April 2026

Programme: BCA (AI, Cloud Computing with DevOps)

Student Name: Muhammed Abdul Gafoor Register Number: **23BCAICD068**

WPPR: **5**

Internal Guide's Name:

MAJOR PROJECT Title:

Multi-Cloud Hybrid Strategy for Smart City

Targets set for the week:

- Complete backup and restore system
- Integrate cloud storage
- Testing and final documentation

Progress/Achievements for the week:

- Completed backup system (database + media files)
- Implemented restore functionality
- Integrated Google Drive API for cloud backup
- Tested all modules (user, admin, authority)
- Fixed bugs and optimized performance
- Completed HLD and LLD documentation

1

22-04-2026

Future Work Plans:

- Final project review
- Viva preparation

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

2

3. Appendix

- Complete source file list:
    - app.py
    - templates/index.html
    - static/app.js
    - static/styles.css
    - data.json
    - main.py (optional CLI)
- API request/response samples:
    - POST /add payload and response
    - GET /checkout response
    - POST /chatbot action/message examples
- Test case checklist and outcomes
- Setup and run instructions
- Known issues and workaround notes
- Additional UI snapshots
- Change log from CLI version to website version

3.1 Weekly report

- Weekly project report for week commencing-02 March

02-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 1

For week commencing 02 March 2026

Programme: BCA (Artificial Intelligence, Cloud Computing, and DevOps)

Student Name: AlenaLatheesh
WPPR: 1
Internal Guide's Name: Mr. Praveen

Register Number: **23BCAICD015**

MAJOR PROJECT Title:

Design and Development of Natural Language Interface for E-commerce Infrastructure

Targets set for the week:

- Develop backend using Flask
- Implement product-related APIs
- Connect CLI with backend
- Implement NLP command detection

Progress/Achievements for the week:

- DevelopedFlask backend with REST APIs
- Implemented endpoints:
  /products
  /products/<id>
- CreatedJSON file for product database
- IntegratedCLI with backend using HTTP requests
- Implemented basic NLP keyword-based command parsing

1

02-03-2026

Future Work Plans:

- Develop cart functionality
- Implement add-to-cart with validation
- Improve NLP command recognition
- Start checkout module

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

2

- Weekly project report for week commencing- 09 March

09-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 2

For week commencing 09 March 2026

Programme: BCA (Artificial Intelligence, Cloud Computing, and DevOps)

Student Name: AlenaLatheesh
WPPR: 2
Internal Guide's Name: Mr. Praveen

Register Number: **23BCAICD015**

MAJOR PROJECT Title:

Design and Development of Natural Language Interface for E-commerce Infrastructure

Targets set for the week:

- Develop cart functionality backend
- Implement add-to-cart with validation
- Build CLI cart viewing flow

Progress/Achievements for the week:

- Implemented the /cart and /add POST endpoints in the Flask backend (app.py) to manage cart state.
- Added backend validation logic to ensure requested quantities are positive integers and that sufficient product stock exists in data.json before adding to the cart.
- Updated the CLI client (main.py) with a cart_flow function that retrieves cart data, maps prices to calculate subtotals, and dynamically displays the final total amount.
- Created helper functions (build_cart_summary and build_overview) to consistently calculate item counts and total costs.

1

09-03-2026

Future Work Plans:

- Develop the final checkout backend logic.
- Implement CLI checkout confirmation flow.
- Expand the NLP backend logic to handle more complex user queries.

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

2

- Weekly project report for week commencing- 16 March

16-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 3

For week commencing 16 March 2026

Programme: BCA (Artificial Intelligence, Cloud Computing, and DevOps)

Student Name: AlenaLatheesh
Register Number: **23BCAICD015**
WPPR: 3
Internal Guide's Name: Mr. Praveen

MAJOR PROJECT Title:

Design and Development of Natural Language Interface for E-commerce Infrastructure

Targets set for the week:

- Develop the final checkout backend logic.
- Implement CLI checkout confirmation flow.
- Expand the NLP backend logic to handle more complex user queries.

Progress/Achievements for the week:

- Built the /checkout endpoint in app.py to process orders, return success messages, and clear the cart state.
- Implemented the user confirmation prompt inside the CLI cart_flow() to finalize the checkout process.
- Built the user profile viewing functionality (profile_flow()) within the CLI.
- Started structuring the core NLP intent-matching engine (build_chat_response) to handle keyword variations for statuses, best values, and delivery help.

1

16-03-2026

Future Work Plans:

- Finalize all chatbot text responses (payment, returns, contact help).
- Conduct end-to-end testing of the CLI client.
- Prepare repository documentation.

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

2

- Weekly project report for week commencing- 23 March

23-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 4

For week commencing 23 March 2026

Programme: BCA (Artificial Intelligence, Cloud Computing, and DevOps)

Student Name: AlenaLatheesh
WPPR: 4
Internal Guide's Name: Mr. Praveen

Register Number: **23BCAICD015**

MAJOR PROJECT Title:

Design and Development of Natural Language Interface for E-commerce Infrastructure

Targets set for the week:

- Finalize all chatbot text responses (payment, returns, contact help).
- Conduct end-to-end testing of the CLI client.
- Prepare repository documentation.

Progress/Achievements for the week:

- Completed the NLP build_chat_response logic, allowing the system to recognize intents like "payment options", "return policy", and "offers".
- Structured the chatbot text outputs into a clean, reusable format featuring titles, messages, and itemized lists for clear CLI reading.
- Executed extensive manual end-to-end testing, simulating various user NLP inputs in the CLI to ensure backend state (data.json) remains synchronized.
- Drafted the initial project documentation (README.md), outlining local setup instructions and the required CLI execution commands.

1

23-03-2026

Future Work Plans:

- Perform final code reviews and optimizations for the CLI build.
- Configure environment variables and .gitignore file.
- Push the final CLI/Backend codebase to the GitHub repository.

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

2

- Weekly project report for week commencing- 30 March

30-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 5

For week commencing 30 March 2026

Programme: BCA (Artificial Intelligence, Cloud Computing, and DevOps)

Student Name: AlenaLatheesh
Register Number: **23BCAICD015**
WPPR: 5
Internal Guide's Name: Mr. Praveen

MAJOR PROJECT Title:

Design and Development of Natural Language Interface for E-commerce Infrastructure

Targets set for the week:

- Perform final code reviews and optimizations for the CLI build.
- Configure environment variables and .gitignore file.
- Push the final CLI/Backend codebase to the GitHub repository.

Progress/Achievements for the week:

- Conducted final code walkthroughs, ensuring all Flask endpoints and NLP parsing logic function without crashing on unexpected inputs.
- Created a .gitignore file to prevent tracking of unnecessary files such as ecom_env/, __pycache__/, .pyc, and .zip archives.
- Finalized the README.md to cleanly document API routes and project structure.
- Successfully initialized Git, committed the final version of the CLI application, and pushed the repository to GitHub.

1

30-03-2026

Future Work Plans:

- Begin researching and planning a web-based GUI (HTML/CSS/JS) to replace the CLI interface.
- Plan API updates to serve front-end dashboard data.

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

2

- Weekly report for week commencing- 13 April

13-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 6

For week commencing 13 April 2026

Programme: BCA (Artificial Intelligence, Cloud Computing, and DevOps)

Student Name: AlenaLatheesh
Register Number: **23BCAICD015**
WPPR: 6
Internal Guide's Name: Mr. Praveen

MAJOR PROJECT Title:

Design and Development of Natural Language Interface for E-commerce Infrastructure

Targets set for the week:

- Develop a web-based GUI (Dashboard) using HTML/CSS.
- Integrate the chatbot directly into the web dashboard interface.
- Create a dedicated endpoint for the web front-end.

Progress/Achievements for the week:

- Created the main index route (/) in Flask using render_template to serve a web-based dashboard.
- Built the front-end layout (index.html and styles.css) featuring product cards with quantity selection, a cart summary, and a profile snapshot.
- Integrated a right-side interactive chatbot drawer directly into the web dashboard, allowing users to interact with the NLP engine visually.
- Added the /dashboard-data API endpoint to supply real-time JSON summaries of products and cart state to the web client.
- Successfully completed all front-end integrations on April 17th.

1

13-04-2026

Future Work Plans:

- Prepare project presentation and live demonstration materials for the final evaluation.

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

2